Toward Accelerated LLM Inference: Porting and Evaluating Diffusion-Based Speculative Decoding on TPU

Aaron Feng[1] Zhongyan Luo[1] Son Nguyen[1] Andy Huang[1] Hao Zhang[1] (Mentor)

[1]Halıcıoğlu Data Science Institute, UC San Diego

Project Website
hpsonnguyen.github.io/tpu-dflash

Problem & Motivation

- LLM decoding is **inherently sequential**: generating n tokens requires n target-model forward passes. Latency grows linearly with output length, making decoding the dominant bottleneck.
- **Speculative decoding:** A lightweight *draft model* proposes candidate tokens; the full *target model* verifies them all in one batched pass. Good drafts ⇒ multiple tokens accepted per verification step.
- **Our contribution:** Port DFlash—a diffusion-based speculative decoding method—from GPU/PyTorch to TPU/JAX inside the `tpu-inference` runtime. Benchmarked on Qwen3-4B across 9 datasets (math, code, chat).
- **TPU advantage:** Verification cost is **flat from K=16 to K=128** (0.97×), while GPU scales to 2.3×. DFlash + TPU is the only combination where *both* draft and verification stay $O(1)$.

Background

Speculative Decoding

- Draft n candidate tokens with a fast model; verify all at once via rejection sampling.
- Key metric: τ = average accepted tokens per draft step. Higher τ = fewer expensive target calls.

Draft Model Approaches

- **N-gram:** Non-neural, fast, but low τ.
- **Eagle3:** 1-layer autoregressive drafter. Good τ, but $O(k)$ sequential passes for k drafts.
- **DFlash:** Diffusion-based *block drafter*—predicts all 16 tokens in **one forward pass** via non-causal attention. Draft cost is $O(1)$.

DFlash Architecture

- 4-layer transformer with **non-causal block attention** (all 16 positions attend bidirectionally).
- Reuses target model's embedding layer and LM head—no separate vocabulary projection.
- Conditioned on target hidden states from layers [1, 9, 17, 25, 33], projected via FC + RMSNorm.
- Trained on only 289K samples; outperforms Eagle3 (1.4M samples) in inference acceleration.

Methods: GPU → TPU Migration

Key Engineering Steps

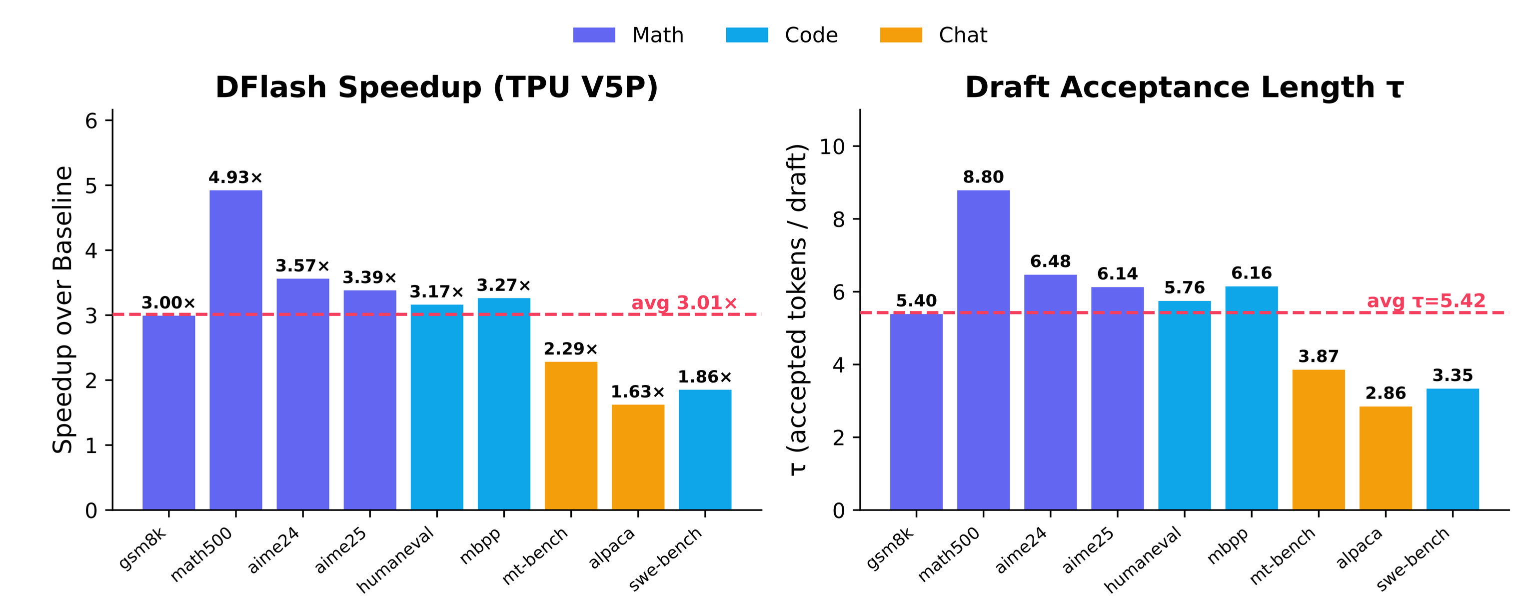
1. **Dual KV cache:** Paged KV (vLLM PagedAttention) for the target model; separate static JAX KV cache with `dynamic_update_slice` for the draft model.
2. **Non-causal attention kernel:** Draft layers routed through TPU `flash_attention` with `causal=False`; target retains ragged paged causal attention.
3. **Sequence-length inflation fix:** `seq_lens` included ∼15 unverified phantom tokens per step, corrupting RoPE embeddings. *Fixing this single bug nearly doubled performance:* τ: 2.49 → 4.48 Speedup: 1.30× → 2.31×
4. **Target hidden-state extraction:** Hidden states from layers [1,9,17,25,33] concatenated and projected via FC + RMSNorm as draft conditioning.
5. **Method registration:** DFlash integrated as a new `"dflash"` branch in `tpu_runner.py` with compilation prewarm support.

The implementation preserves the full DFlash contract with **zero new vLLM dependencies**.

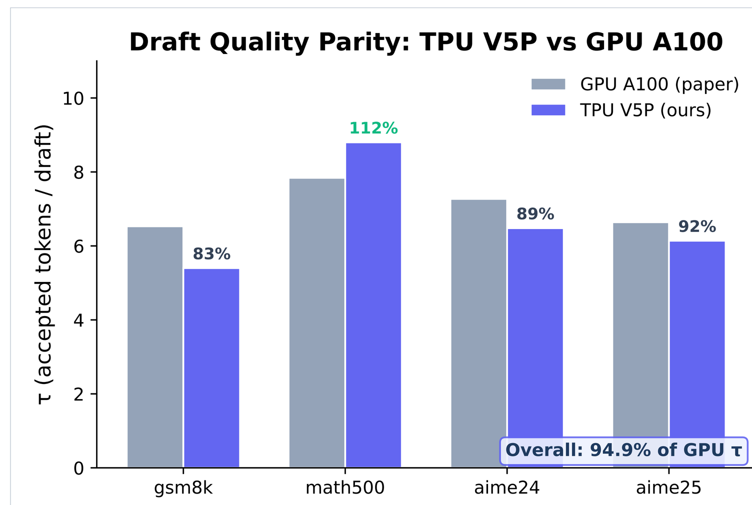
Results

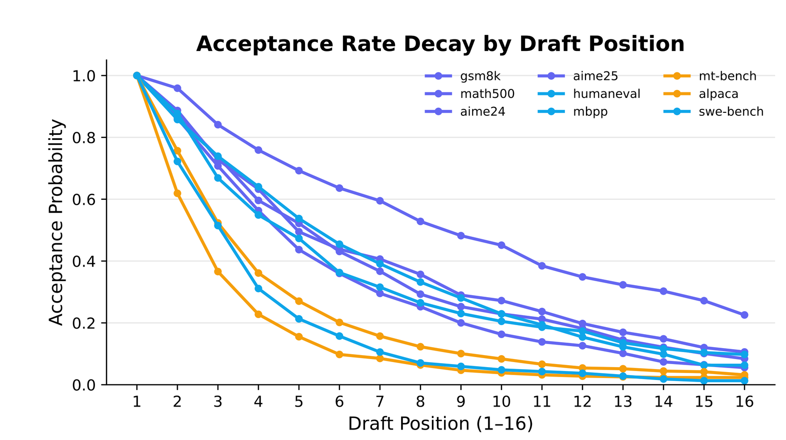
2.31× vLLM pipeline **3.01×** Standalone V5P τ **= 5.42** Avg. accepted

- Benchmarked on Qwen3-4B + DFlash-b16 across **9 datasets** on TPU V5P (4 chips).
- Math: **3.72×** (τ=6.71) | Code: **2.77×** (τ=5.09) | Chat: **1.96×** (τ=3.37).
- Peak: Math500 achieves **4.93×** (τ=8.80), *exceeding* GPU paper τ=7.84.
- TPU achieves **94.9%** of GPU paper τ on math benchmarks.



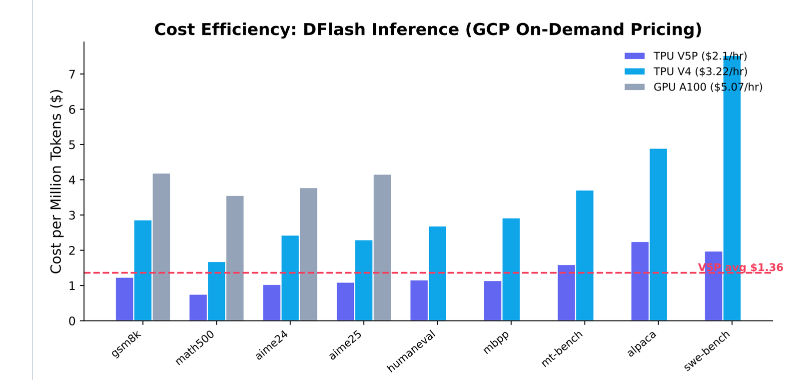
Key Findings

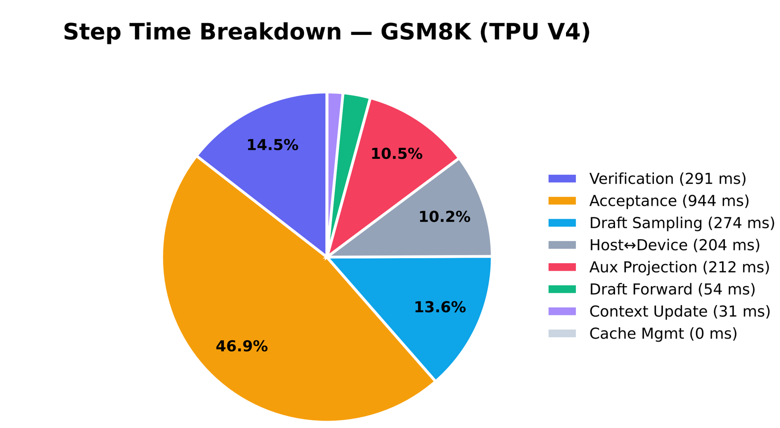




- **Draft quality parity:** 94.9% of GPU τ; Math500 *exceeds* GPU (112%).
- **Task difficulty:** Math maintains >70% acceptance through position 8–10; chat drops below 50% by position 4–5.
- **TPU verification is flat:** K=128/K=16 ratio = 0.97× on TPU vs 2.3× on GPU.

Conclusion & Future Work





Conclusions

- DFlash on TPU: 3.01× standalone, 2.31× pipeline speedup.
- Output quality preserved (bf16 divergence, not correctness errors).
- Pipeline overhead—not model compute—is the remaining bottleneck.
- TPU V5P + DFlash is the most cost-efficient inference option.

Future Work

- **Wider blocks (K=64, 128):** Both draft and verify cost stay flat on TPU.
- **vLLM optimization:** Close standalone–pipeline τ gap (6.67 vs 4.48).
- **LM head fusion:** Two LM-head matmuls account for ∼30% of step time.

References & Acknowledgements

References

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. vLLM: Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[2] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up speculative sampling requires rethinking draft model training. In *arXiv preprint arXiv:2503.01840*, 2025.

[3] Hao Zhang et al. DFlash: Accelerating autoregressive inference with diffusion-based speculative decoding. *arXiv preprint*, 2025. `https://github.com/z-lab/DFlash`.

Acknowledgements: We thank our mentor Hao Zhang and TA Yiming Zhao for their guidance, and the UCSD Halıcıoğlu Data Science Institute for supporting this DSC180B capstone project.